PROGRAMMING & CS FUNDAMENTALS

Design Patterns II: Structural Visual Guide

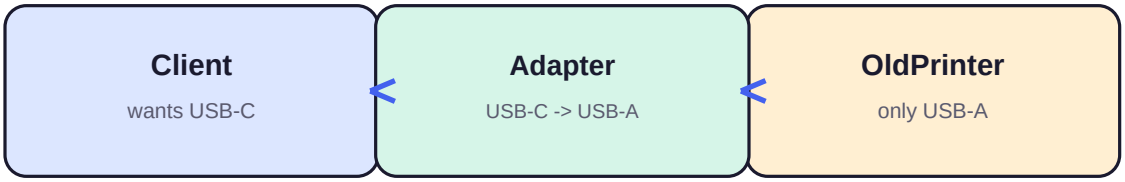
How objects fit together —
five ways to compose interfaces.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Adapter: Make Incompatible Interfaces Fit

The plug converter — wrap it, don't rewrite it.




```
class PrinterAdapter(Printer):      # looks like a Printer
    def __init__(self, old): self.old = old
    def print(self, doc): self.old.print_usb_a(doc)  # translates
```

- Client talks to the TARGET interface; the adapter translates to the ADAPTEE.
- Use when: third-party library, legacy code, or API version mismatch.
- Object adapter (composition) beats class adapter (inheritance) — more flexible.

STEP 2/6

Decorator: Add Behavior in Layers

Wrap the object — each layer adds one responsibility.

total: $2.70 — any combination, no class explosion

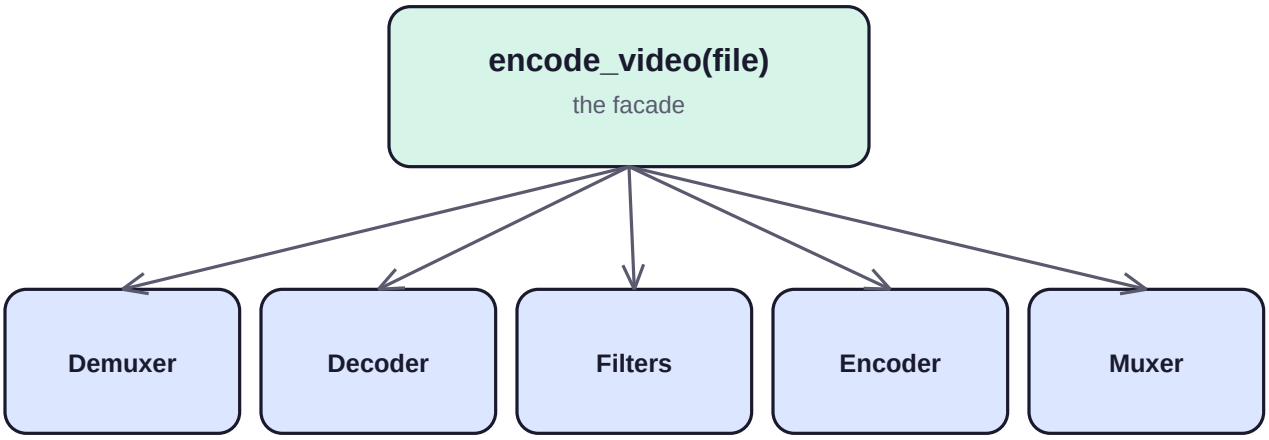
```
class Milk(CoffeeDecorator):
    def cost(self): return self.coffee.cost() + 0.5  # wrap + extend
# vs subclassing: MilkCoffee, SugarCoffee, MilkSugarCoffee... 2^n classes!
```

- Same interface in and out — decorators stack transparently.
- Python's @decorators and Java I/O streams are decorators in the wild.
- Order can matter: encryption-then-compress differs from compress-then-encrypt.

STEP 3/6

Facade: One Simple Door

Hide a messy subsystem behind a clean function.



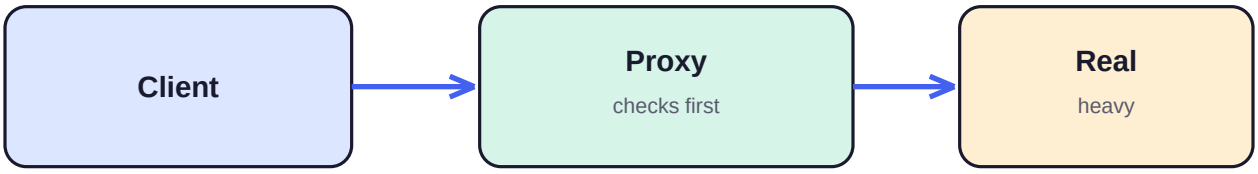

```
def encode_video(f):           # client calls ONE function
    raw = Demuxer(f).split()   # facade orchestrates five subsystems
    ...
```

- Facade doesn't hide the subsystem — power users can still go direct.
- Libraries live here: requests hides sockets, SSL, and redirects behind get().
- One facade per workflow; don't build a god-facade over everything.

STEP 4/6

Proxy: A Stand-In with Control

Same interface, extra gatekeeping — lazy, protective, or remote.



Virtual proxy

lazy-load the heavy object (big images, ORM)

Protection proxy

auth check before the call proceeds

Remote proxy

local stub for a remote object (RPC)

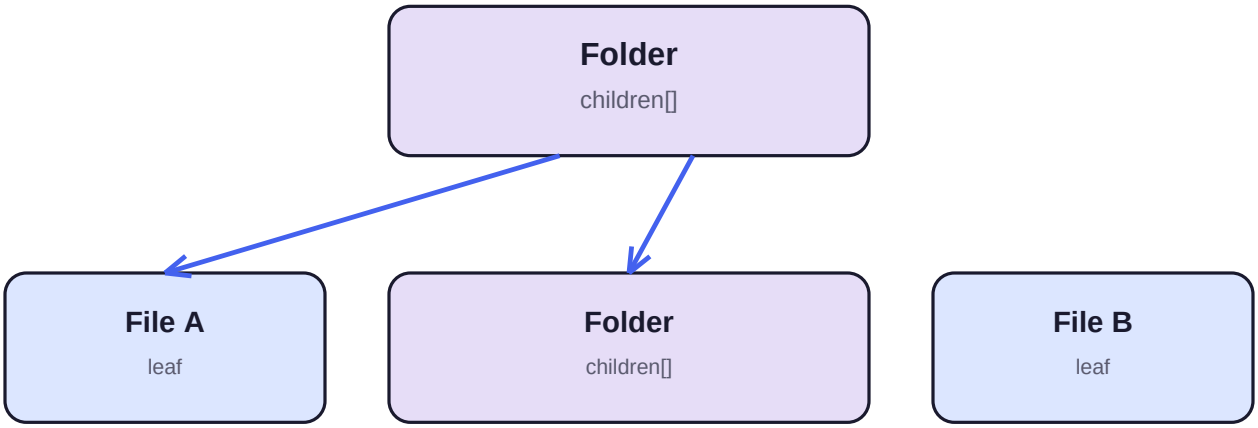
Caching proxy

memoize expensive results

STEP 5/6

Composite: Trees of Uniform Parts

Treat a leaf and a whole subtree the same way.




```
class Node:
    def size(self): ...           # File: its bytes
class Folder(Node):               # Composite: sum of children
    def size(self): return sum(k.size() for k in self.children)
```

- Client code never asks 'file or folder?' — one interface, recursive structure.
- DOM trees, org charts, UI widget hierarchies, arithmetic expressions.
- Price: the interface must fit BOTH leaves and composites — keep it minimal.

STEP 6/6

Structural Chooser

Which assembly pattern for which pain?

Adapter

Incompatible interfaces — wrap and translate

Decorator

Add behavior dynamically — layered wrapping

Facade

Simplify a subsystem — one clean entry point

Proxy

Control access to an object — lazy / guard / remote / cache

Composite

Part-whole hierarchies — leaf and group, one interface

Adapter vs Facade vs Proxy: adapter CHANGES an interface, facade SIMPLIFIES many, proxy KEEPS the interface and adds control.

STEP ✓/✓

Key Takeaways

Structural Patterns — Remember

- Adapter wraps incompatible interfaces (USB-C to USB-A) without rewriting either side.
- Decorator layers behavior by wrapping — same interface in and out, no class explosion.
- Facade gives one simple door over a messy subsystem; power users can still go direct.
- Proxy stands in with control: lazy loading, auth gates, remote stubs, caching.
- Composite treats leaves and groups uniformly — file/folder, DOM, widget trees.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev